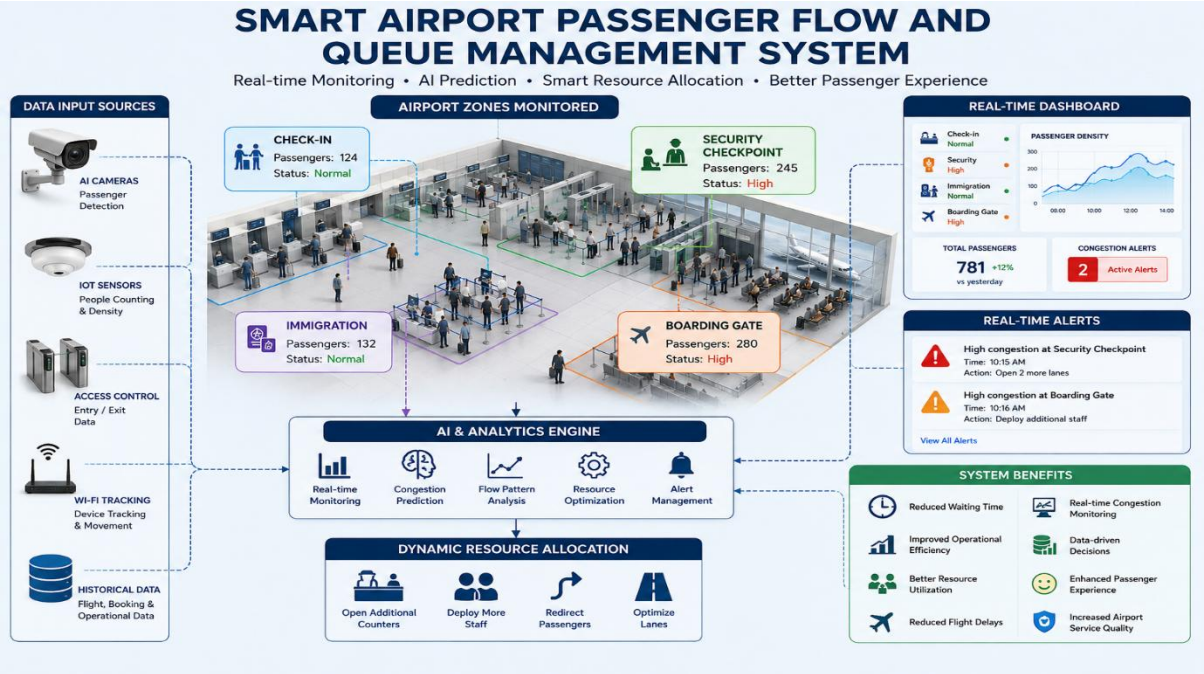
SOFTWARE ENGINEERING ASSESEMENT TOOL CO4 AT2

SMART AIRPORT PASSENGER FLOW AND QUEUE MANAGEMENT SYSTEM



NAME :M.HANIF RIYAZ

REG NO :192521123

COURSE CODE:CSA 1013

SMART AIRPORT PASSENGER FLOW AND QUEUE MANAGEMENT SYSTEM

1. Introduction

The Smart Airport Passenger Flow and Queue Management System is an intelligent software platform designed to monitor and manage passenger movement inside airports. The system uses Artificial Intelligence (AI), Computer Vision, IoT sensors, cloud-based analytics, and real-time monitoring to identify passenger density and predict congestion.

The system monitors important airport areas such as check-in counters, security checkpoints, immigration areas, and boarding gates. It provides real-time information to airport authorities and recommends suitable resource allocation to reduce passenger waiting time and improve airport operational efficiency.

Since the system handles real-time passenger information and AI-based predictions, continuous integration and continuous deployment (CI/CD) is required to ensure that new software updates are

properly tested, secure, reliable, and deployed without affecting airport operations.

---

2. Project Requirements and CI/CD Needs

The major requirements of the Smart Airport Passenger Flow and Queue Management System are:

1. Real-time passenger movement monitoring.
2. Passenger density detection using cameras and IoT sensors.
3. AI-based congestion prediction.
4. Intelligent queue management.
5. Dynamic allocation of airport resources.
6. Real-time congestion alerts.
7. Passenger flow analysis and reporting.
8. Security checkpoint efficiency monitoring.
9. Cloud-based data processing and storage.
10. Dashboard for airport authorities.

CI/CD Requirements

The CI/CD pipeline should:

- Automatically collect and manage source code changes.
- Build the application automatically.

- Perform unit and integration testing.
- Test AI prediction components.
- Check code quality and security.
- Package the application into deployable components.
- Deploy first to a development environment.
- Perform testing in a staging environment.
- Deploy approved versions to production.
- Monitor the application after deployment.
- Provide notifications for successful or failed builds.
- Support rollback if a deployment causes problems.

Because the airport system is a real-time application, deployment should minimize downtime and avoid disrupting passenger monitoring services.

---

3. Proposed CI/CD Pipeline Architecture

The proposed CI/CD workflow is:

Developer → GitHub → Build → Automated Testing → Code Quality & Security → Package → Development → Staging → Approval → Production → Monitoring → Rollback

Pipeline Stages

Stage	Purpose	Suggested Tool
Source Code Management	Store and manage source code	Git + GitHub
Continuous Integration	Automatically trigger pipeline	GitHub Actions
Build	Compile and build application	Maven / npm / Docker
Unit Testing	Test individual modules	JUnit / PyTest
Integration Testing	Test interaction between modules	PyTest / Postman

Python / PyTest

SonarQube

OWASP Dependency-Check

Docker

Kubernetes

Kubernetes

Kubernetes

Prometheus + Grafana

Email / Teams

Kubernetes

---

4. CI/CD Pipeline Workflow

Step 1: Source Code Management

Developers maintain the Smart Airport Passenger Flow and Queue Management System source code using Git.

GitHub is used as the central repository.

The source code may contain:

- Passenger monitoring module
- Queue management module
- AI congestion prediction module
- IoT sensor integration
- Airport dashboard
- Alert generation module
- Database services
- API services

Developers create separate branches for new features and bug fixes.

Example:

main → development → feature branches

When a developer pushes code or creates a pull request, the CI/CD pipeline is automatically triggered.

Step 2: Continuous Integration

GitHub Actions is used to automate the CI process.

Whenever new code is pushed:

1. The repository is checked out.
2. Dependencies are installed.
3. The application is built.
4. Automated tests are executed.
5. Code quality is checked.
6. Security checks are performed.

If any important stage fails, the pipeline stops and the code is not deployed.

---

5. Build Stage

The build stage converts the source code into a deployable application.

For example, the system may contain:

- Front-end dashboard
- Backend APIs
- AI prediction services
- IoT communication services

Docker can be used to package these components into containers.

Advantages of Docker

- Provides a consistent execution environment.
- Reduces deployment problems.
- Makes applications portable.
- Allows independent services to be deployed.
- Simplifies scaling of airport services.

---

6. Automated Testing Strategy

Automated testing is an important part of the CI/CD pipeline because the airport system must provide reliable real-time services.

6.1 Unit Testing

Unit testing checks individual functions and modules.

Examples:

- Passenger count calculation.
- Queue length calculation.
- Congestion level calculation.
- Alert generation.
- Resource allocation logic.

Tool: PyTest / JUnit

6.2 Integration Testing

Integration testing verifies communication between different system components.

Examples:

- Camera system → Computer Vision module.
- IoT sensors → Backend.
- Backend → Database.
- AI prediction service → Dashboard.
- Alert service → Airport authority dashboard.

Tool: PyTest / Postman

6.3 AI Model Testing

The congestion prediction module should be tested using historical and test passenger-flow data.

Testing should verify:

- Prediction accuracy.
- Response time.
- Congestion classification.
- Handling of unusual passenger-flow conditions.
- Stability of predictions.

The AI module should not be deployed if its performance falls below the predefined acceptance criteria.

6.4 Performance Testing

Performance testing verifies whether the system can handle large numbers of passenger records and sensor events.

It should test:

- Number of simultaneous users.
- Number of IoT sensor messages.
- Dashboard response time.
- API response time.
- Passenger density processing speed.

6.5 Security Testing

Security testing should identify vulnerabilities before deployment.

The pipeline can perform:

- Dependency vulnerability scanning.
- API security testing.
- Authentication testing.

- Authorization testing.
- Container security scanning.

Suggested tools: OWASP Dependency-Check and security scanning tools.

---

7. Code Quality Analysis

SonarQube can be integrated into the pipeline to analyze source code.

It checks for:

- Bugs.
- Code smells.
- Duplicated code.
- Maintainability problems.
- Security vulnerabilities.

The pipeline should stop deployment if critical quality or security conditions are not satisfied.

This ensures that only reliable and maintainable software reaches the airport production environment.

---

8. Packaging

After successful testing and quality checks, the application is packaged using Docker.

Each service can be converted into a Docker image.

Example:

Passenger Monitoring Service → Docker Image

AI Prediction Service → Docker Image

Queue Management Service → Docker Image

Dashboard Service → Docker Image

The images are stored in a secure container registry and are then used for deployment.

---

9. Deployment Environments

The CI/CD pipeline uses three major environments.

Environment	Purpose
Development	Used by developers to test new features
Staging	Used for final testing before production
Production	Used by the actual airport system

Development Environment

New code is automatically deployed after successful CI checks.

Developers verify:

- New features.
- API functionality.

- AI model integration.
- Dashboard functionality.

Staging Environment

The staging environment is similar to the production environment.

It is used for:

- Integration testing.
- Performance testing.
- Security testing.
- User acceptance testing.
- Final verification.

Production Environment

After successful staging validation and approval, the application is deployed to the production environment.

The production environment handles actual airport passenger-flow monitoring.

---

10. Deployment Strategy

A Blue-Green Deployment Strategy is proposed for the Smart Airport system.

In this approach, two production environments are maintained:

Blue Environment → Current stable version

Green Environment → New version

The new software is first deployed to the Green environment.

The new version is tested.

If all tests are successful, traffic is redirected from Blue to Green.

If a serious problem occurs, traffic can immediately be redirected back to Blue.

Advantages

1. Minimizes downtime.
2. Provides quick rollback.
3. Reduces deployment risk.
4. Allows the new version to be tested before receiving live traffic.
5. Suitable for critical airport services.

---

11. Security Measures

Security is important because the system processes airport operational data and passenger-flow information.

The CI/CD pipeline should include:

1. Secure authentication.

2. Role-based access control.
3. Encrypted communication using HTTPS/TLS.
4. Secure API authentication.
5. Dependency vulnerability scanning.
6. Container security scanning.
7. Secure storage of passwords and API keys.
8. Secrets management.
9. Access logging.
10. Regular security testing.

Sensitive credentials should never be directly stored inside the source code.

---

12. Real-Time Monitoring

After production deployment, the system should continuously monitor application health.

Prometheus can collect system metrics and Grafana can display monitoring dashboards.

The monitoring system can track:

- CPU utilization.
- Memory usage.
- API response time.
- Application errors.

- Passenger-processing rate.
- IoT sensor status.
- AI service availability.
- Queue monitoring service status.

If abnormal conditions are detected, an alert can be generated.

---

13. Notification Mechanism

The CI/CD system should send notifications to the development and operations teams.

Notifications can be generated when:

- Build succeeds.
- Build fails.
- Tests fail.
- Security checks fail.
- Deployment succeeds.
- Deployment fails.
- Production errors increase.
- Rollback is performed.

Notifications can be sent through email or collaboration platforms such as Microsoft Teams.

---

14. Rollback Strategy

A rollback mechanism is essential because the airport system must remain operational.

If the newly deployed version causes:

- Application failure.
- Incorrect congestion predictions.
- High response time.
- Database problems.
- Security issues.
- Dashboard failure.

the system can automatically or manually roll back to the previous stable version.

With Blue-Green Deployment:

New Version Failure → Switch Traffic Back → Previous Stable Version

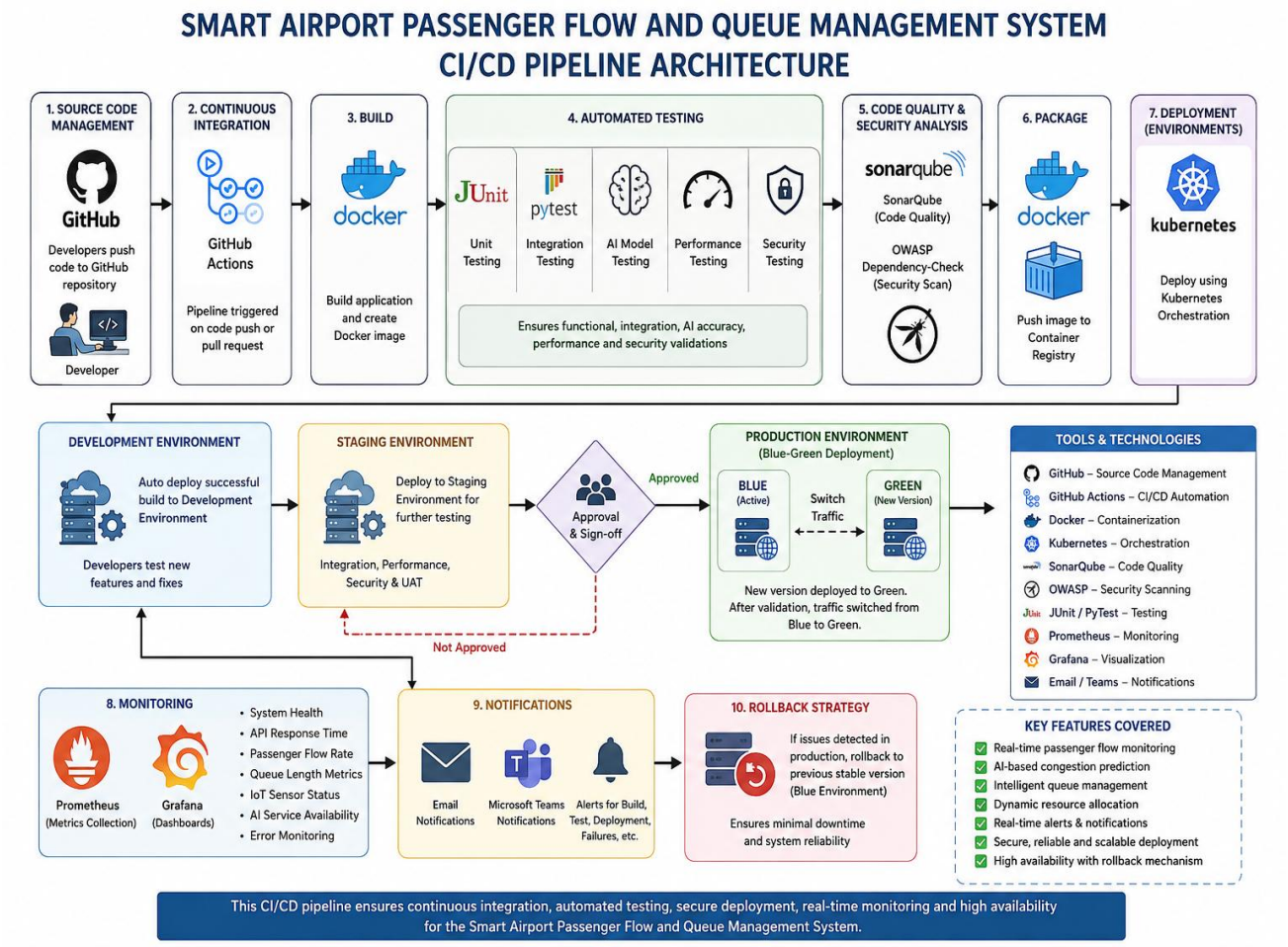
This reduces downtime and maintains continuous airport monitoring.

---

15. CI/CD Pipeline Diagram

The proposed pipeline can be represented as:

SMART AIRPORT CI/CD PIPELINE



16. Tools and Technology Selection

Pipeline Requirement	Tool/Technology	Justification
Source Code Management	GitHub	Centralized source-code management and collaboration
CI Automation	GitHub Actions	Automates build, test, and deployment workflows
Backend Development	Python / Java	Suitable for APIs and application services
AI Processing	Python	Provides libraries for AI and data processing
Unit Testing	PyTest / JUnit	Automates functional testing
API Testing	Postman	Tests backend APIs and service communication
Code Quality	SonarQube	Identifies bugs and code-quality issues

17. Complete CI/CD Workflow

The complete workflow of the Smart Airport Passenger Flow and Queue Management System is:

1. Developer writes or modifies code

↓

2. Code is pushed to GitHub

↓

3. GitHub Actions automatically starts the CI/CD pipeline

↓

4. Application dependencies are installed

↓

5. Application is built

↓

6. Unit tests are executed

↓

7. Integration tests are executed

↓

8. AI model and passenger-flow components are tested

↓

9. Code quality is checked using SonarQube

↓

10. Security vulnerabilities are scanned

↓

11. Docker image is created

↓

12. Application is deployed to Development

↓

13. Application is tested in Staging

↓

14. Final approval is obtained

↓

15. Application is deployed using Blue-Green Deployment

↓

16. Production system is monitored

↓

17. Alerts and notifications are generated when required

↓

18. If a critical failure occurs, the system rolls back to the previous stable version

---

18. Expected Benefits of the CI/CD Pipeline

The proposed CI/CD pipeline provides the following benefits:

1. Faster software delivery.
2. Automated testing and quality assurance.
3. Reduced deployment errors.
4. Improved application reliability.
5. Better security.
6. Reduced production downtime.
7. Quick rollback during failures.
8. Continuous monitoring of the airport system.
9. Consistent deployment environments.
10. Improved maintainability of the software.

---

19. Expected Outcomes for the Smart Airport System

After implementing the proposed system and CI/CD pipeline, the expected outcomes are:

1. Reduced passenger waiting time.

2. Improved airport operational efficiency.
3. Better utilization of airport resources.
4. Enhanced passenger experience.
5. Reduced possibility of flight delays caused by congestion.
6. Real-time passenger-flow monitoring.
7. Data-driven operational decisions.
8. Increased airport service quality.
9. Faster detection of congestion.
10. Reliable and continuously available airport management services.

---

20. Conclusion

The Smart Airport Passenger Flow and Queue Management System requires a reliable and continuously available software platform because airport operations depend on real-time passenger-flow information. A CI/CD pipeline provides an automated approach for integrating, testing, securing, packaging, and deploying the system.

The proposed pipeline uses GitHub, GitHub Actions, automated testing, SonarQube, Docker, Kubernetes, Prometheus, and Grafana. Development, staging, and production environments ensure that new software is

properly validated before reaching the live airport environment. Blue-Green Deployment provides minimal downtime and fast rollback capability.

Therefore, the proposed CI/CD pipeline improves software quality, security, reliability, deployment speed, and operational continuity while supporting the overall objective of reducing airport congestion and improving passenger experience.